Project Development Phase
Model Performance Test

Date	01.04.2026
Team ID	69bf8031c1eab70902258949
Project Name	Enforcing Mandatory Fields Using UI Policies and Migrating Changes with Update Sets
Maximum Marks	

1. Monitoring Mandatory Field Compliance

UI Policies enforce data integrity at the browser level, but they don't affect historical data or records created via background scripts/integrations. You can use Power BI to identify "leaky" processes where mandatory fields remain empty.

- **Data Source:** Connect to the ServiceNow table (e.g., `incident` or `change_request`) using the **ServiceNow Content Pack** or the **REST API**.
- **Key Visualization:** Create a **Gauge Chart** or **Card** showing the "Completeness Rate."
- **DAX Measure for Compliance:**

  $$Compliance \% = \text{DIVIDE}(\text{COUNTROWS(FILTER(Table, NOT(ISBLANK(MandatoryField)))), COUNTROWS(Table)) * 100$$
- **Insight:** If a UI Policy is active but Power BI shows a high volume of empty fields, it indicates that users might be bypassing the UI (e.g., using Import Sets or API calls where UI Policies do not trigger).

2. Visualizing UI Policy Logic

For complex implementations involving hundreds of UI Policies, Power BI can help map out the logic to prevent "Policy Conflicts" (where two policies fight over the same field).

- **Logic Mapping:** Export the `sys_ui_policy` and `sys_ui_policy_action` tables.
- **Conflict Detection:** Use a **Matrix Visual** with **Field Name** on rows and **UI Policy Name** on columns. Use conditional formatting to highlight fields that are marked as "Mandatory" in one policy and "Visible: False" in another.

3. Tracking Migration Health (Update Sets)

Migrating changes from Development to QA/Production can be risky. Power BI can provide a "Deployment Command Center" by pulling data from the `sys_update_set` and `sys_update_xml` tables.

Update Set Velocity & Status

Create a **Funnel Chart** showing the progression of Update Sets:

- **In Progress** (Development)
- **Complete** (Ready for Test)

- **Committed** (Production)

Migration Risk Audit

Metric	Purpose
Collision Count	Number of Update Sets modifying the same UI Policy.
Error Rate	Percentage of Update Sets that resulted in "Preview Errors" during migration.
Backlog Age	How long an Update Set has stayed in "Complete" without being committed.

4. Best Practices for Integration

To get the most out of Power BI for ServiceNow administration:

- **Use the `sys_metadata` Table:** This table tracks all configuration files. Filtering this by "Class = UI Policy" allows you to see who created/modified policies and when.
- **Incremental Refreshes:** Since ServiceNow tables can be massive, configure Power BI to only pull records modified in the last 24 hours to keep your reports fast.
- **Cross-Instance Comparison:** Connect to your **Dev**, **Test**, and **Prod** instances simultaneously in Power BI to ensure the count of active UI Policies matches across all environments.

1. Monitoring Mandatory Field Compliance

UI Policies enforce data integrity at the browser level, but they don't affect historical data or records created via background scripts/integrations. You can use Power BI to identify "leaky" processes where mandatory fields remain empty.

- **Data Source:** Connect to the ServiceNow table (e.g., `incident` or `change_request`) using the **ServiceNow Content Pack** or the **REST API**.
- **Key Visualization:** Create a **Gauge Chart** or **Card** showing the "Completeness Rate."
- **DAX Measure for Compliance:**

  $$Compliance \% = \text{DIVIDE}(\text{COUNTROWS(FILTER(Table, NOT(ISBLANK(MandatoryField)))), COUNTROWS(Table)) * 100$$
- **Insight:** If a UI Policy is active but Power BI shows a high volume of empty fields, it indicates that users might be bypassing the UI (e.g., using Import Sets or API calls where UI Policies do not trigger).

2. Visualizing UI Policy Logic

For complex implementations involving hundreds of UI Policies, Power BI can help map out the logic to prevent "Policy Conflicts" (where two policies fight over the same field).

- **Logic Mapping:** Export the `sys_ui_policy` and `sys_ui_policy_action` tables.
- **Conflict Detection:** Use a **Matrix Visual** with **Field Name** on rows and **UI Policy Name** on columns. Use conditional formatting to highlight fields that are marked as "Mandatory" in one policy and "Visible: False" in another.

3. Tracking Migration Health (Update Sets)

Migrating changes from Development to QA/Production can be risky. Power BI can provide a "Deployment Command Center" by pulling data from the `sys_update_set` and `sys_update_xml` tables.

Update Set Velocity & Status

Create a **Funnel Chart** showing the progression of Update Sets:

- **In Progress** (Development)
- **Complete** (Ready for Test)
- **Committed** (Production)

Migration Risk Audit

Metric	Purpose
Collision Count	Number of Update Sets modifying the same UI Policy.
Error Rate	Percentage of Update Sets that resulted in "Preview Errors" during migration.
Backlog Age	How long an Update Set has stayed in "Complete" without being committed.

4. Best Practices for Integration

To get the most out of Power BI for ServiceNow administration:

- **Use the `sys_metadata` Table:** This table tracks all configuration files. Filtering this by "Class = UI Policy" allows you to see who created/modified policies and when.
- **Incremental Refreshes:** Since ServiceNow tables can be massive, configure Power BI to only pull records modified in the last 24 hours to keep your reports fast.
- **Cross-Instance Comparison:** Connect to your **Dev**, **Test**, and **Prod** instances simultaneously in Power BI to ensure the count of active UI Policies matches across all environments.